

An Analysis of Rectifier neural networks

Rectifier neural networks

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

Rectifier neural networks -- Part 1

$$f(x) = \ln(1 + e^x), f'(x) = \frac{e^x}{1 + e^x} = \frac{1}{1 + e^{-x}} \\ \{ \displaystyle f(x) = \ln(1 + e^x), \quad f'(x) = \frac{e^x}{1 + e^x} = \frac{1}{1 + e^{-x}} \}$$

Rectifier neural networks -- Part 2

$$f(x) = \ln(1 + e^{kx}), f'(x) = \frac{e^{kx}}{1 + e^{kx}} = \frac{1}{1 + e^{-kx}} \\ \{ \displaystyle f(x) = \ln(1 + e^{kx}), \quad f'(x) = \frac{e^{kx}}{1 + e^{kx}} = \frac{1}{1 + e^{-kx}} \}$$

Rectifier neural networks -- Part 3

$$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases} f'(x) = \begin{cases} 1 & x > 0 \\ \alpha e^x & x \leq 0 \end{cases} \\ \{ \displaystyle f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases} \quad f'(x) = \begin{cases} 1 & x > 0 \\ \alpha e^x & x \leq 0 \end{cases} \}$$

Rectifier neural networks -- Part 4

Non-differentiability at zero (however, it is differentiable anywhere else, and the value of the derivative at zero can be chosen to be 0 or 1 arbitrarily). Not zero-centered: ReLU outputs are always non-negative. This can make it harder for the network to learn during backpropagation, because gradient updates tend to push weights in one direction (positive or negative). Batch normalization can help address this. ReLU is unbounded. Redundancy of the parametrization: Because ReLU is scale-invariant, the network computes the exact same function by scaling the weights and biases in front of a ReLU activation by k , and the weights after by $1/k$. Dying ReLU: ReLU neurons can sometimes be pushed into states in which they become inactive for essentially all inputs. In this state, no gradients flow backward through the neuron, and so the neuron becomes stuck in a perpetually inactive state (it "dies"). This is a form of the vanishing gradient problem. In some cases, large numbers of neurons in a network can become stuck in dead states, effectively decreasing the model capacity and potentially even halting the learning process. This problem typically arises when the learning rate is set too high. It may be mitigated by using "leaky" ReLU instead, where a small positive slope is assigned for $x < 0$. However, depending on the task, performance may be reduced.

Rectifier neural networks -- Part 5

where $\Phi(x) = P(X \leq x)$ is the cumulative distribution function of the standard normal distribution and $\operatorname{erf}(z)$ is the error function. This activation function is illustrated in the figure at the start of this article. It has a "bump" with negative derivative to the left of $x < 0$. It serves as the default activation for many transformer models such as BERT. The same paper also introduces a few faster approximations for GELU. The first approximation follows from an approximation for $\Phi(x)$:

Rectifier neural networks -- Part 6

The second, less-precise approximation uses the sigmoid (logistic) function as $f(x) \approx x \cdot \operatorname{sigmoid}(1.702x)$, whose formula resembles SiLU (see below).

Rectifier neural networks -- Part 7

$$f(x) = x \Phi(x) = x \cdot \frac{1}{2} [1 + \operatorname{erf}(x/\sqrt{2})] \\ \{ \displaystyle f(x) = x \Phi(x) = x \cdot \frac{1}{2} [1 + \operatorname{erf}(x/\sqrt{2})] \}$$

Rectifier neural networks -- Part 8

where $\tanh(x)$ is the hyperbolic tangent, and $\operatorname{softplus}(x)$ is the softplus function. Mish was obtained by experimenting with functions similar to Swish (SiLU, see above). It is non-monotonic (has a "bump") like Swish. The main new feature is that it exhibits a "self-regularizing" behavior attributed to a term in its first derivative.

Rectifier neural networks -- Part 9

In the context of artificial neural networks, the rectifier or ReLU (rectified linear unit) activation function is an activation function defined as the non-negative part of its argument, i.e., the ramp function:

Rectifier neural networks -- Part 10

This equation is foundational to the model's capacity; while the attention mechanism determines the relationships between tokens, the ReLU-based FFN performs the majority of the numerical computation and houses the bulk of the model's parameters. The efficiency and scalability of this rectified framework triggered a global technological revolution, enabling the development of Large Language Models that have had a profound economic impact. The industrial response to this

An Analysis of Rectifier neural networks

Rectifier neural networks

architecture—including the massive expansion of AI-specific hardware and the birth of the generative AI sector—has positioned the Transformer as a cornerstone of 21st-century infrastructure. During the post 2017 period of rapid AI advancement, the rectified linear unit function has been key to achieving increased model performance and scaling due to the fact that it zeros out responses that are immaterial for a given stimuli, preventing them from accumulating in massive scale models. It is the complete silencing of the parts of the model found to be stimuli-irrelevant during learning that allows for scaling. As the stimuli-irrelevant proportion of the model becomes more massive, these highly numerous connections within the model would inevitably accumulate during scaling no matter how small each individual response is. Therefore, the rectified linear unit function, with its absolute zeroing property, enabled the scaling to hundred billion parameter models and beyond. Early Transformer scaling giants like GPT-3 (2020) and Falcon-180B (2023) relied on the rectified linear unit function explicitly, while successors such as GPT-4 (2023) and Llama 3 (2024) utilized smoother variants like GELU or SwiGLU. These variants were used to improve training stability while fundamentally preserving the rectified principle of zeroing low responses. At the centre of modern artificial intelligence ReLU and its variants maintain absolute zero response across the bulk of the model at any one time, while maintaining approximately linear responses for stimuli-relevant connections enabling high performance on each specific cognitive task. This feature of activation sparsity has been critical for massive scaling and performance gains of AI models right up to the present day.

Rectifier neural networks -- Part 11

In these formulas, a , b and x_c are hyperparameter values which could be set as default constraints $a = 1$, $b = 2$ and $x_c = 1.25643$, as done in the original work.